

Logistics

- Attendance - mark using SAFE (between 3:25 - 3:40)
 - For any issues: write to safe@cse.iitb.ac.in
- Accessing documents (iitb login)
- Extra lab sessions (every Saturday 2-4 pm).
- Not having laptops, Additional slots? — talk to me at the end.

Lecture 3: Variables, data types, expressions

What we have seen so far

- Turtle graphics commands like, `forward(n)`, `left(n)`
- `repeat(n){}` — iteration,
- `cin`, `cout`
- How all data is represented using numbers
- Numbers are stored in binary (sequences of 0/1)

How numbers are written in a computer?

- A computer has a large set of capacitors
 - 8 GB RAM means $2^{36} \approx 6 \times 10^{10}$ capacitors
 - Each capacitor represents a bit (0 or 1)
 - Low charge represents 0
 - High charge represents 1
- Once you represent 0 and 1, you can represent anything.
- Binary number system
- Electrical circuits are designed to perform basic operations like addition, multiplication, copying.

Binary number system

- A sequence $a_n a_{n-1} \dots a_1 a_0 a_{-1} \dots a_{-k}$ of 0s and 1s
- Will represent the number
$$a_n 2^n + a_{n-1} 2^{n-1} + \dots + a_1 2^1 + a_0 2^0 + a_{-1} 2^{-1} + \dots + a_{-k} 2^{-k}$$
- Converting decimal integer v to binary
 - Divide v by 2, remainder gives a_0
 - Repeat previous step with the quotient to get $a_1, a_2, \dots$
- Converting fraction f to binary
 - If $f > 0.5$, $a_{-1} = 1$ and 0 otherwise.
 - Similarly other bits...

Positive/negative numbers

- Typically number of capacitors used for an integer
 - 8, 16, 32, 64
- One of the bits (leftmost) indicates sign
- Numbers stored from -2^{31} from $2^{31}-1$
- Two's complement
 - For $0 \leq x \leq 2^{31}-1$, x represents x
 - For $2^{31} \leq x \leq 2^{32}-1$, x represents $2^{32} - x$
 - See next slide for examples.

Two's complement for three bits

Bits	Unsigned value	Signed value
000	0	0
001	1	1
010	2	2
011	3	3
100	4	-4
101	5	-3
110	6	-2
111	7	-1

int nSides

```
int nSides;
```

- C++ will typically designate some 32 capacitors in your computer for the variable nsides.
- The bit pattern stored in it will be interpreted as having a sign and a magnitude.
- 10000000000000000000000000000000011001 will mean $-2^{31}+25$

```
unsigned int nSides;
```

- 32 capacitors will be given, but the bit pattern in it will be interpreted as 32 bit binary number.
- 10000000000000000000000000000000011001 will mean $2^{31}+25$

Bit, Byte, word

- Bit = 1 binary “digit”, (one number = 0 or 1)
- byte = 8 bits
- half-word = 16 bits
- word = 32 bits
- double word = 64 bits

- “one byte of memory” = memory capable of storing 8 bits = 8 capacitors.

Representing real numbers

- Use analogue of scientific notation: *significand* $\times 10^{\text{exponent}}$
 - e.g. 6.022×10^{23}
- Same idea, but significand, exponent are in binary, thus number is:
 - *significand* $\times 2^{\text{exponent}}$
- “Single precision”:
 - store significand in 24 bits (7-8 decimal digits),
 - exponent in 8 bits.
 - Fits in one word!
- “Double precision”: store significand in 53 bits, exponent in 11 bits.
 - Fits in a double word!
 - 53 bits of significand = 16-17 decimal digits
- More implementation details later

Exercises

- Suppose I want to represent 8 digit telephone numbers.
 - I should use 8 / 16 / 32 / 64 bit signed / unsigned representation.
- What is roughly the largest number that can be represented using 64 bits?
- If we represent real numbers with 24 bits significand and 8 bit exponent
 - What is the largest possible number ?
 - What is the smallest possible positive number ?

Summary

- Numbers are represented by sequence of 0s and 1s
- The same sequence may mean one number as an unsigned integer, a signed integer, or a floating point number, or a character
- The capacitors only store high or low charge, they are not “aware” that the charge represents numbers.
- So long as we remember what type of number we are storing, there will be no problem.
- As a user, you don't need type/read binary numbers.
- C++ will convert binary numbers to decimal system while printing
- C++ will accept numbers typed in decimal by you and itself convert it to binary for use on the computer.
- But you should know (roughly) what range of numbers can be stored in k bit unsigned / signed / floating formats

Reserving memory for storing numbers

- Before you store numbers, you must explicitly reserve space for storing them.
 - “space” : region of memory
- This is done by a “**variable definition**” statement.
- **variable**: name given to the space you reserved.
- “**Value of a variable**”: value stored in the variable
- You must also state what kind of values will be stored in the variable: “**data type**” of the variable.

Variable creation / definition

- Statement form:
`data-type-name variable-name;`
- Example from chapter 1:
`int nSides;`
- `int` : data type name. Short for “integer”.
- Reserve space for storing integer values, positive or negative, of a “standard” size.
- Standard size = 32 bits on most computers.
- Two’s complement representation will typically be used.
- `nSides` : name given to reserved space, or the created variable.

Variable names/identifier

- Sequence of 1 or more letters, digits and the underscore “_” character
 - Should not begin with a digit
 - Some words such as **int** cannot be used as variable names. Reserved by C++ for its own use.
 - Case matters. ABC and abc are distinct identifiers
 - Space not allowed inside variable name
- **Examples:** nsides, telephone_number, x, x123, third_cousin
- **Not valid:** #sides, 3rd_cousin, 3 rd cousin
- **Recommendation:** use meaningful names, describing the purpose for which the variable will be used.

Variable names/identifier

- **unsigned int** : Used for storing integers which will always be positive.
 - 1 word will be allocated.
 - Ordinary binary representation will be used.
- **char** : Used for storing characters or small integers.
 - 1 byte will be allocated.
 - ASCII code of characters is stored.
- **float** : Used for storing real numbers
 - 1 word will be allocated.
 - IEEE FP representation, 8 bits exponent, 24 bits significand.
- **Double** : Used for storing real numbers
 - 2 words will be allocated.
 - IEEE FP representation, 11 bits exponent, 53 bits significand.

Examples

```
unsigned int telephone_number;
```

```
float mass, acceleration;
```

- You can define several variables in same statement as above.
- Keyword long : says, “I need to store bigger or more precise numbers, so give me more than usual space.”

```
long unsigned int cryptographic_password;
```

- Likely 64 bits will be allocated.

```
long double more_precise_acceleration;
```

- Likely 96 bits will be allocated

Variable initialization

- A value can be stored in a variable at the time of creation

```
int i=0;
```

```
float vx=1.0, vy=2.0e5, weight;
```

- `i`, `vx`, `vy` given values as well as defined.
- `2.0e5` is how you write 2.0×10^5
- Although the computer uses binary, you write in decimal.

```
char command = 'f';
```

- `'f'` is a “character constant”. It represents the ASCII value of the quoted character.

Reading values into variables

```
cin >> nSides;
```

- Can read into several variables one after another

```
cin >> vx >> vy;
```

- User expected to type in values consistent with the type of the variable into which it is to be read.
- “Whitespace” = space characters, tabs, newlines, typed by the user are ignored.
- newline/enter key must be pressed after values are typed
- If you read into a char type variable, the ASCII code of the typed character gets stored.

```
char command;
```

```
cin >> command;
```

- If you type the character ‘f’, its ASCII value, 102, will get stored.

Printing variables on the screen

- General form: `cout << variable-name;`
- To print newline, use `endl`.
- Additional text can be printed by enclosing it in quotes.
- Many things can be printed by placing `<<` between them.

```
cout << "Position: " << x << ", " << y << endl;
```

- Prints the text “Position: “, then values of variables `x`, `y` with a comma between them and a newline after them.
- If you print a char variable, then the content is interpreted as an ASCII code, and the corresponding character is printed.

```
char command = 'G', command2 = 97;
```

```
cout << command << command2; // Ga will be printed.
```

Assignment statement: storing a value into a variable defined earlier

- Statement form: **variable = expression;**

s = u*t + 0.5 * a * t * t;

- Expression : formula involving constants or variables, almost as in mathematics.
- Execution: The value of expression is calculated and stored into the variable which appears on the left hand side of the = symbol.
 - If expression contains variables they must have been assigned a value earlier – this value will be used to calculate the value of the expression.
 - If **u**, **a**, **t** have values 1, 2, 3 respectively, $1*3 + 0.5 * 2 * 3 * 3 = 12$ will be stored in **s**.
 - The value present in **s** before the execution of the statement is lost or “overwritten”.
 - The values present in other variables including **u**, **a**, **t** does not change when the above statement executes.

Rules regarding expressions

- Multiplication must be written explicitly.
- `S = u t + 0.5 a t t; //` not legal
- Multiplication, division have higher precedence than addition, subtraction
- Multiplication, division have same precedence
- Addition, subtraction have same precedence
- Operators of same precedence will be evaluated left to right.
- Parentheses can be used with usual meaning.
- Spaces can be put between operators and values as you like

`s=u*t+0.5*a*t*t;      s = u*t + 0.5*a*t*t;      // both OK`

Examples

```
int x=2, y=3, p=4, q=5, r, s, t, u;
```

```
r = x*y + p*q;
```

```
// 2*3 + 4*5 = 26
```

```
s = x*(y+p)*5;
```

```
// 2*(3+4)*5 = 70
```

```
t = x - y + p - u;
```

```
//wrong as u not defined earlier
```

Example with division

```
int x=2, y=3, z=4, u;
```

```
u = x/y + z/y;
```

- What will this do?
- If dividend and divisor are both integers, then result is the quotient, i.e. remainder is ignored.
- u becomes: $2/3 + 4/3 = 0 + 1 = 1$
- If you divide an int variable by zero, it has undefined behaviour.

Rules for storing numbers of one type into variable of another type

- C++ does the “best possible”.

```
int x; float y;
```

```
x = 2.5;
```

```
y = 123456789;
```

- x will become 2, since it can hold only integers. Fractional part is dropped.
- 123456789 cannot be precisely represented in 24 bits, so something like 1.234567 e 9 will get stored.

Evaluating “A op B” when A, B have different types

- If A,B have different data types: then they will be converted to “more expressive” data type among the two:
 - int/short/unsigned int are less expressive than float/double
 - shorter types are less expressive than longer.
- The operation will then be performed, and the result will have the more expressive type.
- For example, if you multiply int with float, you will get float.

Examples

```
int nsides=100,
```

```
int iangle1, iangle2;  
iangle1 = 360/nsides;  
iangle2 = 360.0/nsides;
```

```
float fangle1, fangle2;  
fangle1 = 360/nsides;  
fangle2 = 360.0/nsides;
```

- $360/\text{nsides}$: both are integer so integer division is used.
- Result $360/100 = 3$
- 3 is stored in iangle1
- $360.0/\text{nsides}$: numerator is double, so denominator also converted to double.
- Result $360.0/100.0 = 3.6$
- But iangle2 is int, so integer part, 3, stored.
- $360/\text{nsides}$ evaluates to 3, stored in fangle1
- $360.0/\text{nsides}$ evaluates to 3.6, stored in fangle2

Implication of limited precision

- `float w, y=1.5, avogadro=6.022e23;`
- `w = y + avogadro;`
- “Actual sum” : 602200000000000000000001.5
- Sum will have to be stored in variable `w` of type `float`.
 - But `w` can only accommodate 23 bits, or about 7 digits.
 - Thus all digits after the 7th will get truncated.
 - To 7 digits of precision `avogadro` is same as `y+avogadro`.
- `w` will receive the truncated value, i.e. `avogadro`.
- This addition has no effect!

Program example

```
main_program{
    double centigrade, fahrenheit;
    cout <<"Give temperature in Centigrade: ";
    cin >> centigrade;
    fahrenheit = centigrade * 9 / 5 + 32;
    cout << "In Fahrenheit: " << fahrenheit
        << endl;    // newline.
}
// Do we need to write 9.0?
```

Where expressions can appear

- Expressions can appear on the right hand side of an assignment.
- As a rule: expressions can appear wherever a plain number can

```
double u=1, t=2, a=3, v = u+a*t, s = (u+v)*t/2;
```

- Initialization happens left to right.

```
int x=5*y, y=2; // incorrect
```

```
cout << u+a*t; // OK if u, a, t have values
```

```
forward(u*10); // forward u*10 pixels
```

A useful operator: % for finding remainder

- $x \% y$ evaluates to the remainder when x is divided by y .
same as $x - x/y*y$
 x, y must be integer expressions.

- Example

```
int n=12345678, d0, d1;
```

```
d0 = n % 10;
```

```
d1 = (n / 10) % 10;
```

- $d0$ will equal the least significant digit of n , 8.
- $d1$ will equal the second least significant digit of n , 7.

Exercise

What are the results of evaluating the following expressions

- $(1+2)/4$
- $(1.0+2)/4$

Write a program that reads time in seconds and prints it as x hours, y minutes, z seconds.

- Note that one hour has 60 minutes, and one minutes has 60 seconds
- You will find it useful to remember the properties of integer division and the % operator.

What we discussed

- The assignment statement
- Rules about how arithmetic happens when an expression contains numbers of different types.
- Also to be noted is that real numbers are represented only to a fixed number of digits of precision in the significand, and so adding very small values to very large values may have no effect.
- What we did not discuss but you should read from the book: overflow, representation of infinity.



Re-assignment

- Same variable can be assigned again.
- When a variable appears in a statement, its value at the time of the execution gets used.

```
int p=3, q=4, r;  
r = p + q;           // 7 stored into r  
cout << r << endl;   // 7 printed  
r = p * q;           // 12 stored into r  
cout << r << endl;   // 12 printed
```

- = is the assignment operator, not equality.

Why do we need Re-assignment

- Let us try a mental exercise
- You need to find the sum of following numbers (without using pen)
- 3, 7, 2, 5, 9, 8
- Perhaps you went over the numbers left to right, and kept adding them
- When you were at the last number,
 - did you remember what was the sum of first three numbers?
 - Or did you erase it?
 - Perhaps you allocate some space in the mind, where you keep the current sum, and keep overwriting it.

Why do we need Re-assignment

- You need to find the sum of following numbers (without using pen)
- 3, 7, 2, 5, 9, 8
- Perhaps you allocate some space in the mind, where you keep the current sum, and keep overwriting it.
- Doing this saves the “memory space”
- More importantly, this makes it possible to use **repeat** loop. Because we always add the next number to the same “memory location”.

Why do we need Re-assignment

- Program to read 6 numbers and find their sum

```
int curr_sum = 0;
int next, new_sum;

repeat(6){
    cin >> next;
    new_sum = curr_sum + next;
    curr_sum = new_sum;
}
```

```
int curr_sum = 0,
int next;

repeat(6){
    cin >> next;
    curr_sum = curr_sum + next;
}
```

An interesting assignment expression

```
int p = 12;  
p = p + 1;
```

- Follow usual rule for evaluation:
 - First evaluate the value on the right hand side.
 - Then store the result into the LHS variable.
- So 1 is added to 12, the value of p
- The result, 13, is then stored in p.
- Thus p finally becomes 13.

“`p = p + 1`” is nonsensical in mathematics.

“=” in C++ is different from “=” in math.

Repeat and reassignment

spiral demo

```
forward(3); left(60);  
forward(6); left(60);  
forward(9); left(60);  
forward(12); left(60);  
forward(15); left(60);  
forward(18); left(60);  
forward(21); left(60);  
:  
forward(150); left(60);
```

Can we do this using **repeat**?

Repeat and reassignment

```
#include <simplecpp>
main_program{
    turtleSim();
    int i = 1;
    repeat(50){
        forward(3*i);
        left(60);
        i = i + 1;
    }
    getClick();
}
```

- First iteration:
 - *i* changes to 1.
- Second iteration:
 - *i* changes to 2....
- 50th iteration:
 - *i* changes to 50.
- Fundamental idiom: **sequence generation**.
- What does this draw?
 - “Hexagonal spiral”

Other sequences can also be generated

- Can you make i take values 1, 3, 5, 7, ...?
- Can you make i take values 1, 2, 4, 8, 16,
- Both can be done by making slight modifications to previous program.

Another idiom: accumulation

```
// Program to read 10 numbers and add them.
main_program{
    int term, s = 0;
    repeat(10){
        cin >> term;
        s = s + term;
    }
    cout << s << endl;
}
// values read get “accumulated” into s
// Accumulation happens here using +
// We could use other operators too.
```

Composing the two idioms: program to calculate $n!$

```
main_program{
    int i=1, n;    cin >> n;
    int nfac=1;
    repeat(n){
        nfac = nfac * i; // multiplied by 1 to n
        i = i + 1; // i goes from 1 to n
    }
    cout << nfac << endl; // n! printed
}
```

Some additional operators

- The fragment “ $i = i + 1$ ” appears very frequently, and so can be abbreviated as “ $i++$ ”.
- **$++$: increment operator** Unary
- Similarly we may write “ $j --$ ” which means “ $j = j - 1$ ”
- **$--$: decrement operator**

Exercise

- What does the following program do?

```
unsigned int n=7589, m=0;
repeat(4){
    m = 10*m + (n % 10);
    n = n/10;
}
```

m=9 n=758

m=90+8 = 98 , n= 75

m=980+5 = 985 n= 7

m=9850+7= 9857, n=0;

What we discussed

- The value of a variable can be changed (reassigned).
- Assignments such as $i = i+1$ or $j = j*2$ are allowed.
 - They are useful for generating sequences
 - Once you can generate sequences, you can compute expressions such as $n!$
 - Other uses will be seen later
- Because assignments such as $i = i+1$ and $j = j*2$ are very frequently needed, operators such as $++$ and $*=$ have been provided.



Blocks and Scope

Code inside {} is called a **block**

- Blocks are associated with repeats, but you may create them otherwise too.
- You may declare variables inside any block.

New summing program:

- The variable `term` is defined close to where it is used, rather than at the beginning. This makes the program more readable.
- But the execution of this code is a bit involved.

```
// The summing program
// written differently

main_program{
    int s = 0;
    repeat(10){
        int term;
        cin >> term;
        s = s + term;
    }
    cout << s << endl;
}
```

How definitions in a block execute

Basic rules

- A variable is defined/created every time control reaches the definition.
- All variables defined in a block are destroyed every time control reaches the end of the block.
- “Creating” a variable is only notional; the compiler simply starts using that region of memory from then on.
- Likewise “destroying” a variable is notional.
- New summing program executes exactly like the old, it just reads different (better!).

Scope and Shadowing

- Variables defined outside a block can be used inside the block, if no variable of the same name is defined inside the block.
- If a variable of the same name is defined, then from the point of definition to the end of the block, the newly defined variable gets used.
- The new variable is said to “**shadow**” the old variable.
- The region of the program where a variable defined in a particular definition can be used is said to be the **scope** of the definition.
- Why do we care:
 - In a single English language document you might write “Let x denote” in several places, with the understanding that the x on page 5 is different from the x on page 37.
 - If you do not have an intervening “Let x denote ..” the two xs might be same.
 - Something similar happens while writing large programs

Example

```
main_program{  
    int x=5;  
    cout << x << endl;    // prints 5  
    {  
        cout << x << endl; // prints 5  
        int x = 10;  
        cout << x << endl; // prints 10  
    }  
    cout << x << endl; // prints 5  
}
```

What if `int x = 10;` was `x = 10;` ?

Concluding Remarks

- Variables are regions of memory which can store values.
- Variables have a type, as decided at the time of creation.
- Choose variable names to fit the purpose for which the variable is defined.
- The name of the variable may refer to the region of memory (if the name appears on the left hand side of an assignment), or its value (if the name appears on the right hand side of an assignment).

More remarks

- Expressions in C++ are similar to those in mathematics, except that values may get converted from integer to real or vice versa and truncation might happen.
- Truncation may also happen when values get stored into a variable.
- Sequence generation and accumulation are very common idioms.
- Increment/decrement operators and compound assignment operators also are commonly used.

More remarks

- Variables can be defined inside any block.
- Variables defined outside a block may get shadowed by variables defined inside.

